



Single Number (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Solution with XOR
- Code

Problem Statement

In a non-empty array of integers, every number appears twice except for one, find that single number.

Example 1:

Input: 1, 4, 2, 1, 3, 2, 3
Output: 4

Example 2:

Input: 7, 9, 7
Output: 9

Try it yourself

Try solving this question here:



```
1 class SingleNumber {
2     public static int findSingleNumber(int[] arr) {
3         // TODO: Write your code here
4         return -1;
5     }
6
7     public static void main( String args[] ) {
8         System.out.println(findSingleNumber(new int[]{1, 4, 2, 1, 3, 2, 3}));
9     }
10 }
```



Run

Save

Reset



Solution

One straight forward solution can be to use a **HashMap** kind of data structure and iterate through the input:

- If number is already present in **HashMap**, remove it.
- If number is not present in **HashMap**, add it.
- In the end, only number left in the **HashMap** is our required single number.

Time and space complexity Time Complexity of the above solution will be $O(n)$ and space complexity will also be $O(n)$.

Can we do better than this using the **XOR Pattern**?

Solution with XOR

Recall the following two properties of XOR:

- It returns zero if we take XOR of two same numbers.
- It returns the same number if we XOR with zero.

So we can XOR all the numbers in the input; duplicate numbers will zero out each other and we will be left with the single number.

Code

Here is what our algorithm will look like:

 Java

 Python3

 C++

 JS

```
1 class SingleNumber {
2     public static int findSingleNumber(int[] arr) {
3         int num = 0;
4         for (int i=0; i < arr.length; i++) {
5             num = num ^ arr[i];
6         }
7         return num;
8     }
9 }
10 public static void main( String args[] ) {
11     System.out.println(findSingleNumber(new int[]{1, 4, 2, 1, 3, 2, 3}));
12 }
13 }
```

Run

Save

Reset



Time Complexity: Time complexity of this solution is $O(n)$ as we iterate through all numbers of the input once.

Space Complexity: The algorithm runs in constant space $O(1)$.

[< Back](#)[✓ Mark As Completed](#)[Next >](#)[Introduction](#)[Two Single Numbers \(med.\)](#)

